c:\hpc-23-feb-2026\.\24\eg181.cpp

```
#include<oneapi/tbb.h>
#include<vector>
#include<chrono>
#include<iostream>
using namespace std;
using namespace oneapi::tbb;
int main()
{
vector<int64_t> v(10000000,10);
auto start=chrono::high_resolution_clock::now();
int64_t total=parallel_reduce(blocked_range<size_t>(0,v.size()),
0LL,
[&v](blocked_range<size_t> &range,int64_t acc_sum){
for(auto i=range.begin();i!=range.end();++i)  // i is not an iteration type object
{
acc_sum+=v[i];
}
return acc_sum;
},
[](int p,int q)->int{return p+q;}
);
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<duration.count()<<" milliseconds"<<endl;
cout<<"Total "<<total<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\24\eg182.cpp

```
#include<iostream>
using namespace std;
int main()
{
int x,y,z;
auto i=[&](){
x=10;
y=20;
z=x+y;
};
i();
cout<<x<<","<<y<<","<<z<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\24\eg183.cpp

```
#include<iostream>
#include<vector>
#include<cstdint>
#include<numeric>
using namespace std;
int main()
{
vector<int64_t> v(100000,10);
int64_t sum=accumulate(v.begin(),v.end(),0LL);
cout<<sum<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\24\eg184.cpp

```
#include<iostream>
#include<vector>
#include<cstdint>
#include<numeric>
using namespace std;
int main()
{
vector<int64_t> v={1,2,3,4,5,6,7,8,9,10};
int64_t sum=accumulate(v.begin(),v.end(),0LL,[](int64_t p,int64_t q)->int64_t{
cout<<p<<","<<q<<endl;
return p+q;
});
cout<<sum<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\24\eg185.cpp

```
#include<iostream>
#include<vector>
#include<cstdint>
#include<numeric>
using namespace std;
int main()
{
vector<int64_t> v={1,2,3,4,5,6,7,8,9,10};
int64_t sum=accumulate(v.begin(),v.end(),1LL,[](int64_t p,int64_t q)->int64_t{
cout<<p<<","<<q<<endl;
return p*q;
});
cout<<sum<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\24\eg186.cpp

```
#include<iostream>
#include<vector>
#include<cstdint>
#include<numeric>
using namespace std;
template<typename T>
class binary_operation_plus
{
public:
T operator()(T p,T q)
{
cout<<p<<","<<q<<endl;
return p+q;
}
};
int main()
{
vector<int64_t> v={1,2,3,4,5,6,7,8,9,10};
int64_t sum=accumulate(v.begin(),v.end(),1LL,binary_operation_plus<int64_t>());
cout<<sum<<endl;
return 0;
}
```